基于关键变量状态感知的定向灰盒模糊测试

摘要

定向模糊测试是一种有效的软件测试方法，它可将模糊测试过程导向用户定义的目标关注位点，从而发现与这些位点相关的漏洞。然而，即便生成的测试用例覆盖了目标位点附近的代码，复杂漏洞仍无法被触发。若仅关注覆盖新边的测试用例，与目标相关的程序状态会被忽略，这会导致对目标的测试不充分，且无法捕获复杂漏洞。

本文提出了一种名为 CSFuzz 的新型定向模糊测试解决方案，该方案考虑了与目标相关的程序状态。首先，CSFuzz 通过静态分析从程序中提取与目标位点相关的关键变量；随后，CSFuzz 监控这些关键变量的运行时值，并通过自适应划分变量值的范围来推断与目标相关的程序状态，这使其能将触发目标位点附近新状态的有效种子存入状态语料库。最后，CSFuzz 采用动态调度技术指导模糊测试过程，以选择不同的语料库并对种子进行优先级排序，从而确保对目标位点进行更充分的测试。我们实现了 CSFuzz 的原型系统，并在 2 个基准测试程序和经过广泛模糊测试的真实软件上对其进行了评估。评估结果表明，CSFuzz 在漏洞检测能力上优于当前最先进的模糊测试工具，最大加速比达到 219%。此外，CSFuzz 还发现了 4 个新漏洞，其中 2 个已分配 CVE 编号。

关键词： 模糊测试；定向测试；软件测试

1 引言

模糊测试是一种用于发现漏洞的高效自动化程序测试技术，它对工业界 [1–4] 和学术界 [5–8] 均产生了重大影响。通过变异选定的种子生成大量测试用例来执行被测程序（PUT），模糊测试在程序出现崩溃时能够简便且有效地发现程序漏洞 [9]。根据测试方式的不同，模糊测试技术可分为黑盒测试 [10]、白盒测试 [11–13] 以及灰盒测试 [4]、[14] 三类从程序内部获取的信息量。灰盒模糊测试借助轻量插桩获取部分运行时程序信息，在效率和开销之间取得平衡。因此，灰盒模糊测试已获得广泛研究与应用 [1]、[4]、[6]、[14]。

尽管灰盒模糊测试已得到广泛研究与应用，但其对所有代码区域一视同仁，并未专门针对关键目标位点进行测试。为解决这一问题，Böhme 等人于 2017 年提出了 AFLGo [15]，这是定向灰盒模糊测试（Directed Greybox Fuzzing，DGF）的先驱。定向灰盒模糊测试的目标是将更多时间用于抵达并测试目标位点，而非将资源浪费在无关的代码区域。为引导模糊测试过程指向目标位点，DGF 需要确定种子执行路径与目标位点之间

的接近程度。在模糊测试过程中，测试用例的能量分配基于其得分。因此，与传统灰盒模糊测试最大化代码覆盖率不同，定向灰盒模糊测试能更快地抵达并测试被测试程序（PUT）中的目标位点。凭借这些特性，定向灰盒模糊测试在补丁测试 [16]、验证静态分析报告 [17] 以及崩溃复现 [18] 等场景中更具适用性，且表现更为出色。

为了更有效地到达目标位点并触发对应的漏洞，DGF 相关方法做出了诸多努力。鹰眼（Hawkeye）[19] 提出了多样化的距离策略来评估种子的执行轨迹，以此解决 AFLGo 的距离偏差问题。FishFuzz [20] 利用与目标函数的距离作为基于向量的指标。WindRanger [21] 引入了基本块偏差的概念，并利用该概念来衡量距离。泰坦（Titan）[22] 结合程序中目标之间的相关性来优化输入生成。AFLRun [23] 为每个目标维护一个额外的原始映射，以整合路径多样性，并为目标实现无偏的能量分配。

然而，当前的 DGF 在漏洞发现方面的效果未达预期。这是因为在正常情况下，执行 PUT 需要达到特定的程序状态才能渗透代码并抵达目标位点。此外，即便目标代码被执行，也未必会触发相应的漏洞，因为触发条件的满足需要特定的程序状态。例如，假设测试用例 S 在运行时被被测程序（PUT）执行，若其未触发新的边覆盖，则会被丢弃；但如果 S 产生了有助于检测目标漏洞的新程序状态，那么它就是一个有价值的测试用例。为了提升 DGF 在发现这类漏洞时的有效性，我们需要聚焦于这些关键的程序状态。

首先，**什么是程序状态**？本质上，程序会将数据存储在变量中，在程序执行的任意时刻，这些存储的内容都被称为程序状态。然而，程序中的变量数量众多，若将所有变量都纳入考量，会带来无法接受的性能开销，进而阻碍模糊测试工作的推进。因此，要在有效性和性能开销之间取得平衡，就需要聚焦于状态的子集。此外，**DGF 应该跟踪哪些程序状态**？为了提高触发潜在漏洞的有效性，我们需要更多关注与目标位点相关的状态。首先，我们考虑变量与目标位点的距离及其可达性。在静态单赋值语言中，每个变量只能被赋值一次。这意味着在触发漏洞之前，一些对目标位点有影响但距离较远的变量，在使用前会被赋值为新变量。由此可见，距离崩溃位点较远的变量对目标程序状态的影响微乎其微，甚至没有影响。同时，我们排除那些没有路径可达目标位点的变量，因为这些变量无法对目标位点产生影响。其次，我们考虑目标位点相关内存的位置。记录特定状态的变量值通常存储在内存中。其中部分值会直接或间接影响分支。此外，用内存不安全语言编写的软件中，违反内存安全的漏洞往往具有安全隐患 [24] [25]。AddressSanitizer（ASan）是一款内存错误检测工具，能够记录无效的内存读写操作 [26]。因此，我们认为与 ASan 检测相关的变量值与目标相关状态的关联更为紧密，考虑这些相关状态有助于触发对应的漏洞。

Listing 1: 简化的动机示例

```
static int fill_buffer_resample(lame_internal_flags * gfc,
    sample_t * outbuf,...)
{
    int  BLACKSIZE, xvalue;
    int  *inbuf_old;
    ......
```

```
    BLACKSIZE = filter_l + 1;
    if (gfc->buffer_resample == 0)
    {
        ......
        y = inbuf_old[BLACKSIZE + j2]; // bug location
        xvalue += y
        ......
        outbuf[k] = xvalue;
    }
    else
    {
        ......
    }
}
```

为识别与目标相关的程序状态并指导 DGF，我们提出了 CSFuzz——一种感知关键变量状态的定向灰盒模糊测试方法。首先，通过静态分析，我们根据变量与目标位点的距离以及可达性提取相关变量。随后，我们利用 ASan 注释作为标识来识别关键变量。在运行时，我们监控这些关键变量的值，并基于额外记录的初始值将其值自适应地划分为区间。这使得我们能够结合额外反馈捕获程序状态行为，作为代码覆盖率的补充。我们使用额外的状态语料库存储发现新状态的测试用例，从而更全面地探索程序的目标区域。我们还采用动态种子调度策略以提升性能。

我们将 CSFuzz 部署在通用模糊测试框架 AFL [4] 上，以验证其有效性和可扩展性。为评估 CSFuzz 的性能，我们选取 Unibench [27] 和 Magma [28] 作为基准测试集，并将其与当前主流的模糊测试工具进行对比，包括 AFL [4]、AFL++ [1]、FishFuzz [20]、AFLGo [15]、WindRanger [21] 以及 AFL++ 的 CmpLog 模式。我们对每个程序开展了 10 轮测试，并计算平均值以评估崩溃复现和漏洞发现能力。实验结果表明，相较于 AFL、AFL++、FishFuzz、AFLGo、WindRanger 以及 AFL++ 的 CmpLog 模式，CSFuzz 在触发目标崩溃方面平均实现了 152%、107%、119%、86%、88% 和 81% 的加速。此外，CSFuzz 在 gpac [29] 和 libxml2 [30] 等广泛测试的开源软件中发现了此前未知的漏洞，并成功分配了两个 CVE 编号。

我们做出了以下贡献：

- 我们提出了利用关键变量跟踪程序状态的概念以及一种识别关键变量的静态分析方法。
- 我们提出了一种状态反馈机制，以帮助 DGF 高效地探索目标。
- 我们设计了一种种子调度策略以提高 DGF 的效率。
- 我们实现了 CSFuzz 的原型，将其与最先进的模糊测试方法进行对比，并验证了其在真实世界基准测试程序上的优越性。

2 动机示例

在本节中，我们以来自 lame 的一段代码片段作为示例，阐述并探讨其他定向模糊测试方法在这类场景中查找漏洞时存在的局限性（第2.1节）。此外，我们还研究了 CSFuzz 如何助力成功检测这些漏洞（第2.2节）。

2.1 模糊测试场景

列表1展示了一段处理缓冲区数据的代码片段，其中包含一个潜在的错误。该代码会处理 `inbuf_old` 和 `outbuf` 中的元素，只有 `buffer_resample` 等于零时，才会满足进入错误区域的条件。

假设在定向模糊测试活动开始前，包含该漏洞的目标位点已确定。模糊测试过程会利用代码覆盖率将有价值的种子保留在语料库中。为提升定向性，DGF 会通过考量与目标位点距离更近的种子来对其进行优先级排序。然而在某些情况下，那些拥有新程序状态但未提供新代码覆盖率的测试用例，即便它们对推进模糊测试进程更有帮助，也会被模糊器丢弃。

我们来设想这样一个场景：一个 DGF 首次生成了测试用例 A，该用例执行到第 7 行并实现了新的边覆盖。尽管该用例不满足进一步深入代码的条件，但由于代码覆盖率反馈机制 [4]，测试用例 A 仍被保留在语料库中。随着模糊测试的持续进行，假设新的测试用例 B 以不同的 `buffer_resample` 值执行到了第 7 行。测试用例 B 会创建一个新的程序状态，但无法提供任何新的边覆盖反馈，因此被模糊测试工具丢弃。不过，这个状态揭示了影响目标位点的变量中存在新的行为，这实际上具有很高的研究价值。

同样地，当模糊测试生成首个到达目标位点的测试用例 C 时，这条带有边覆盖反馈的新种子会被保留。然而，由于第 10 行不存在越界访问，错误未必一定会发生。若新的测试用例 D 在第 10 行生成了不同的值，尤其是在 `BLACKSIZE + j2` 发生变化的情况下，会形成与触发漏洞高度相关的程序状态。但由于没有新的边覆盖反馈，该种子会被丢弃，而非存入语料库。这会导致错失进一步模糊测试的机会，也使得目标区域的测试不够充分。

2.2 解决方案

捕捉程序的重要行为，并且更全面地测试目标点，二者都是至关重要的。这其中的挑战在于，既要识别并只关注程序中的关键变量，又要避免在无关领域产生过多的资源消耗以及状态爆炸问题。

在 CSFuzz 中，我们会根据变量与目标位点的距离以及可达性对其进行筛选，选择那些与目标位点更相关的变量。此外，我们会基于 ASan sanitizer 的指示，重点关注与触发漏洞关联度更高的变量。基于此，我们识别出一组有助于触发与目标相关漏洞的变量，例如程序中的 `buffer_resample` 变量以及 `BLACKSIZE + j2` 变量。

为了实现更全面的程序测试，我们在被测试程序（PUT）执行过程中监控关键变量的运行时值。通过对这些变量的取值范围进行自适应划分，我们可以保留那些触发新程

序状态的种子。即使未触发新的边但出现了新的程序状态，我们也能将对应的测试用例保留在测试语料库中，以便后续进行进一步测试。因此，当 `buffer_resample` 的值或 `BLACKSIZE + j2` 发生变化时，这些值会被捕获并视为触发不同的程序行为。相应的测试用例将被保留以进行进一步测试，从而提高触发与该目标位点相关漏洞的可能性。

3 CSFuzz 方法

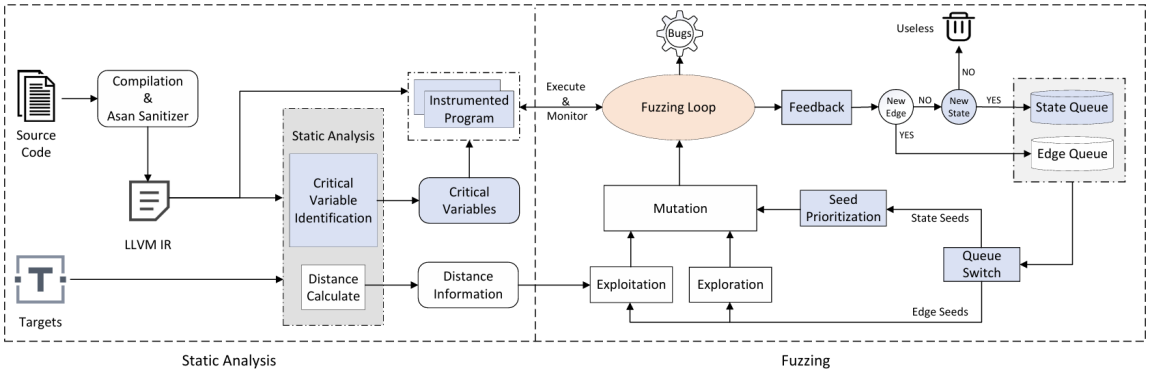

图 1: CSFuzz 工作流程概述

图1展示了 CSFuzz 的完整工作流程，该流程包含两个主要部分：静态分析和模糊测试。在 CSFuzz 的静态分析阶段，源代码被编译为 LLVM IR 中间文件。随后，通过协同结合给定的目标关注位点，识别并筛选出程序中的关键变量。同时，参考 FishFuzz [20] 的方法计算距离信息，优先选择一组更接近目标的种子，以引导模糊测试朝着目标位点推进。除了像其他模糊测试工具一样通过插桩获取边覆盖率外，CSFuzz 还通过插桩来跟踪关键变量，以监控被测程序的运行时状态。

随着模糊测试循环持续进行，CSFuzz 基于反馈信息对关键变量的范围进行动态划分。当测试用例触发新的边覆盖时，该测试用例会被添加到边覆盖语料库中。若未出现新的边覆盖，但变量值处于此前未覆盖的范围，则将其添加到状态语料库。随后，CSFuzz 会为下一个循环选择种子，并制定策略来确定后续阶段种子的选取来源。对于边语料库，CSFuzz 的运行方式与普通 DGF 类似，会在探索阶段与利用阶段之间进行适当切换。而对于状态语料库，CSFuzz 会对种子进行优先级排序，并确定其选取顺序。

在接下来的章节中，我们将详细解释以下问题：CSFuzz 如何通过静态分析提取关键变量（第3.1节）；如何划分变量值的范围以监控程序的运行状态（第3.2节）；如何应用语料库选择策略（第3.3节）；以及如何调度种子以确定队列优先级（第3.4节）。

3.1 关键变量识别

在静态分析阶段，识别程序中的关键变量是 CSFuzz 的主要目标。提取过多的变量会导致运行时向状态语料库中添加过多的种子，从而降低模糊测试的效率。相反，如果在提取过程中忽略了关键变量，则可能导致对重要程序状态的探索不足。在这两方面之间取得适当的平衡至关重要。为了解决这一挑战，我们提出了两种策略。策略 1 用于识

别与目标位点密切相关的变量，策略 2 则聚焦于与 ASan 检查相关的变量。同时满足两种策略标准的变量被视为候选关键变量。随后，我们筛选候选变量并获取特定于 PUT 的关键变量。我们的分析在 LLVM IR 层面进行，LLVM IR 是一种静态单赋值语言，程序中的每个变量仅可被赋值一次。

策略 1. 对于给定程序 p 和目标位点 ts，程序 p 中满足策略 1 的变量定义如下：

$$S_1(p) = \{v \in ALLV(p) \mid (\exists t \in ts)[calldist(F(v), F(t)) < h \wedge isreachable(v,t)]\}, \tag{1}$$

其中 $S_1(p)$ 表示程序 p 中满足策略 1 的变量；$ALLV(p)$ 指代程序 p 中的所有变量；$F(v)$ 表示变量 v 所在的函数；$F(t)$ 表示目标 t 所在的函数；$calldist()$ 用于计算两个函数之间的最短调用距离；h 为阈值；$isreachable(v,t)$ 表示变量 v 是否可通过最短函数调用路径到达目标 t。例如，若函数 A 中的指令 I 调用了函数 B，则两个函数之间的最短调用距离为 1；若函数 B 中存在调用函数 C 的指令，且函数 A 中无直接调用 C 的指令，则 A 与 C 之间的最短调用距离为 2。若变量 v 位于函数 A 中，目标 t 位于函数 B 中，当包含 v 的基本块存在一条通往包含调用指令 I 的基本块的路径时，我们认为 $isreachable(v,t)$ 为真。

在策略 1 中，若目标集合 ts 中存在目标 t，使得变量 v 所在函数与目标 t 所在函数之间的调用距离在阈值范围内，且 v 可到达函数内的调用位点，则 v 被判定为满足策略 1。换言之，策略 1 的目标可解释为：筛选出距离目标位点更近的变量。这是因为这类变量与目标的关联性更强，对触发相关漏洞的贡献更大。此外，CSFuzz 利用 SVF 分析工具 [?] 处理间接调用问题，同时可选取其他函数作为策略 1 的补充，从而提升方法的鲁棒性与可扩展性。

```
cond.true:                                  ; preds = %for.body127
  %316 = load i32*, i32** %inbuf_old
  %317 = load i32, i32* %BLACKSIZE
  %318 = load i32, i32* %j2
  %add133 = add nsw i32 %317, %318
  %idxprom134 = sext i32 %add133 to i64
  %arrayidx135 = getelementptr inbounds i32, i32* %316, i64 %idxprom134
  %319 = ptrtoint i32* %arrayidx135 to i64
  %320 = lshr i64 %319, 3
  %321 = add i64 %320, 2147450880
  %322 = inttoptr i64 %321 to i8*
  %323 = load i8, i8* %322
  %324 = icmp ne i8 %323, 0
  br i1 %324, label %325, label %331

325:                                        ; preds = %cond.true
  %326 = and i64 %319, 7
  %327 = add i64 %326, 3
  %328 = trunc i64 %327 to i8
  %329 = icmp sge i8 %328, %323
  br i1 %329, label %330, label %331

330:                                        ; preds = %325
  call void @__asan_report_load4(i64 %319) #13
  unreachable

331:                                        ; preds = %325, %cond.true
  %332 = load i32, i32* %arrayidx135
  br label %cond.end
```

图 2: 带 ASan 插桩的 LLVM IR

策略 2. 在满足策略 1 的变量基础上，满足策略 2 的变量定义如下：

$$S_2(v) = \{v \in S_1(p) \mid rel(v, asan) \vee rel(getop(v), asan)\}, \tag{2}$$

其中 $S_2(v)$ 表示满足策略 2 的变量；$asan$ 指代程序中用于运行时错误检测的内存安全检测工具标识；rel 对应相关性检查；$getop(v)$ 用于获取变量 v 对应指令的操作数。

换言之，策略 2 的目标可解释为选择与 ASan 检查相关的变量。满足策略 2 的变量更有可能包含与目标相关的程序状态，并触发对应的漏洞。

如图 2 所示，假设这些变量均满足策略 1，虚线框内的代码表示由 sanitizer 插桩的漏洞检测代码。变量%319 受到 ASan 的监控，并且%319 派生自程序中的%arrayidx135。因此，我们认为%arrayidx135 满足策略 2。同理，%332 以%arrayidx135 作为其操作数，由于%arrayidx135 受到 ASan 监控，我们同样认为%332 满足策略 2。

基于策略 1 与策略 2，我们筛选出了与目标位点关联性更强、且具有更高漏洞触发潜力的候选关键变量。为进一步提升效率，我们从这些候选变量中选取指针类型与整数类型的变量。该筛选依据源于如下观察：程序状态通常由整数值表示，而漏洞往往与指针相关。

算法 1 展示了从程序中筛选关键变量的流程。在获取目标所在的函数集合 fs 后（第 1 行），我们遍历程序 P 中的函数 f 与 fs 中的函数 ft，并计算从 f 到 ft 的调用距离（第 2–4 行）。若 f 到函数 ft 的距离小于阈值 h（默认值为 3），则识别 f 中的调用位点 cs（第 5–6 行）。若函数 f 中的变量 v 存在可达该调用位点的路径，则认为其满足策略 1（第 7–9 行）。此外，若变量 v 还与 ASan 相关，则认为其同时满足策略 1 与策略 2（第 15–17 行）。最后，通过过滤指针类型与整数类型变量，得到程序中的关键变量（第 20 行）。

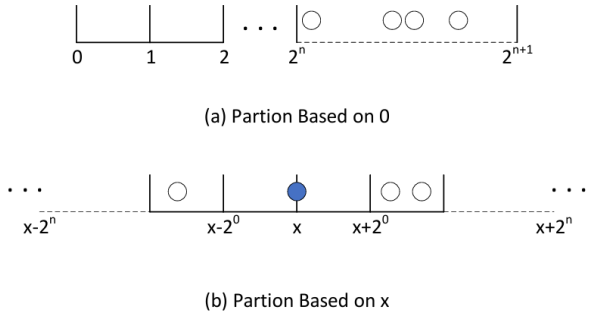

图 3: 变量区间划分方式

3.2 状态反馈

为捕获程序运行时状态，我们对程序中的关键变量执行额外插桩。在 AFL 中，基本块会被插桩处理，并使用共享内存记录当前执行种子的边覆盖信息。AFL 同时使用原始映射（virgin map）存储所有已执行输入的覆盖信息。与 AFL 类似，我们为每个关

Algorithm 1 关键变量识别算法

Require: 目标集合 Ts，插桩程序 P
Ensure: 关键变量集合 CVs

```
fs ← GetFunc(Ts)
for f in P do
    for ft in fs do
        dis ← Calldist(f, ft)
        if dis < h then
            cs ← CallSite(f, ft)
            for v in f do
                if reachable(v, cs) then
                    v1 ← v1 ∪ v
                end if
            end for
        end if
    end for
end for
for v in v1 do
    if rel(v, asan) ∨ rel(getop(v), asan) then
        CVs ← CVs ∪ v
    end if
end for
CVs ← PointerOrInteger(CVs)
```

键变量分配唯一 ID，从而能够在运行时将变量值记录到共享内存的对应位置。为实现统一性并平衡性能开销，我们通过插桩将反馈数据标准化为 32 位。对于低于 32 位的数据，执行零位扩展至 32 位；对于高于 32 位的数据，截断高位并保留低 32 位。该方式虽为粗粒度，但对模糊测试十分有效。

随后，模糊测试器从共享内存读取数据，以获取被测程序（PUT）的运行时信息。利用该运行时信息，需要判断当前测试用例是否触发了新的、有价值的程序状态。每个关键变量的值都可能携带重要的程序状态信息，但跟踪所有状态并不现实。因此，为避免状态爆炸，我们尝试将变量划分为不同区间。若变量值落在之前未记录过的区间内，则认为其触发了新状态。

一种简单的划分方式是采用 AFL 针对数值的二次幂存储机制，例如 $[1,2)$、$[2,4)$、$[4,8)$ 等区间。然而，这种直接划分可能导致监控不准确。如图 3(a) 所示，不同变量具有不同的区间分布，当变量集中在一个较大区间内时，变量状态检测将失效。为解决该问题，我们实现了一种自适应方法，为每个变量分配不同的区间，从而实现更精确的状态检测。在运行时，我们为每个变量记录一个初始值。当变量值首次出现时，使用对应值初始化初始值。如图 3(b) 所示，变量后续的区间划分基于初始值 x 进行，从而能够为不同变量自适应分配区间。

尽管部分变量的分布不可预测，但在运行时探索变量的不同行为可作为边覆盖的有效补充。对于具有常被用于表示特定状态的分布模式的变量，我们利用初始值确定其区间分布。在 CSFuzz 中，该方法能够更有效地捕获行为变化。若测试用例的执行未触发

新的边覆盖，但其追踪覆盖了状态原始映射中此前语料库未覆盖的任意新位，则表明变量值落在了新的区间内。这种情况下，我们认为其触发了新的程序状态。随后，状态原始映射会被更新，该测试用例将被存入状态语料库。这种通过自适应划分变量区间以判断状态的方法，不仅避免了状态爆炸，还能确保对关键变量进行更精确的状态检测。

3.3 语料库选择

为更好地维护触发新边或新状态的种子，我们使用两个语料库来存储有价值的种子。当触发新的边覆盖时，种子被存入语料库 C_1；若未触发新边覆盖，但触发了新的程序状态，则种子被存入语料库 C_2。通过平衡覆盖探索与状态探索，我们帮助 DGF 更快地抵达目标位点并触发对应漏洞。为在模糊测试的下一轮迭代中高效选择种子，我们使用概率 p 来决定从哪个语料库中选取。

$$C = \begin{cases} C_1, & r < p \\ C_2, & \text{else} \end{cases} \tag{3}$$

若在 $[0,1]$ 内生成的随机数 r 小于 p，则在下一轮中从 C_1 中选择种子；否则从 C_2 中选择。概率 p 在运行时每隔固定轮次根据选择不同语料库的奖励值通过 δ 进行动态更新。

$$p = \varphi(p + \delta) \tag{4}$$

$$\delta = a \cdot \frac{m_1}{N_{C_1}} - b \cdot \frac{m_2}{N_{C_2}} \cdot (N_{C_1} + N_{C_2}) \tag{5}$$

其中 a 和 b 为超参数；m_1 表示过去若干轮中选择语料库 C_1 的平均奖励值，奖励值定义为通过变异 C_1 中种子而新增到 C_1 中的测试用例数量。m_2 与 m_1 类似，对应语料库 C_2；N_{C_1} 和 N_{C_2} 分别为两个语料库中的种子数量。种子数量更少的语料库意味着对应种子的新增相对稀少，因此我们在新增种子时让奖励值更加敏感。此外，φ 用于将概率 p 约束在 $[0.1, 0.9]$ 之间，在保证概率合理性的同时避免极端情况。即使倾向于某一个语料库，仍有概率选择另一个语料库以获得奖励，从而逐步调整概率 p。

在实际策略中，我们倾向于根据语料库的奖励调整 p。假设从 C_1 中选择的种子能产生新增至 C_1 的有效测试用例，直观上应提高 C_1 的奖励值，因为存在仍待测试的新代码区域。计算奖励时，我们只考虑被添加到对应语料库的种子数量。其原因在于：若选择 C_1 产生了被加入 C_2 的有效测试用例，这些用例的作用是探索程序状态而非新代码区域，因此不应计入 C_1 的奖励。

CSFuzz 基于动态概率选择语料库，使其既能在定向阶段探索新代码区域、辅助引导模糊测试向目标位点逼近，又能充分探索程序状态，对目标位点开展更充分的测试。

3.4 种子优先级调度

选中的代码覆盖语料库工作方式与其他 DGF 方法类似，分为探索阶段与利用阶段。我们使用固定概率在代码覆盖语料库的探索与利用阶段之间进行切换。在探索阶段，

CSFuzz 专注于充分的覆盖探索，以降低陷入局部最优的风险。在利用阶段，对于每个目标，CSFuzz 会为距离目标更近的种子集合分配更高的选择概率。在下一轮种子选择中，优先级越高的种子被选中的概率越大。

在选择状态语料库时，状态语料库中的种子能够触发与目标位点相关的新状态，代表在目标位点附近执行的种子。因此，我们主要从两个方面对语料库内的种子进行优先级排序。(1) 优先选择所覆盖关键变量的模糊测试次数较少的种子。在关键变量测试次数的统计中，如果种子 s 覆盖了关键变量 a 和 b，且我们选择种子 s 进行模糊测试，则变量 a 和 b 的测试次数各加一。(2) 在当前状态语料库中，优先选择最近加入队列的种子。

种子 x 被选中的概率 $P(x)$ 计算如下：

$$P(x) = \frac{1-a}{\min(\xi(x)) \cdot \sum_{i=1}^{n} \frac{1}{n(v_i)}} + a \cdot \frac{t_x}{\sum_{i=1}^{m} t_i}, \tag{6}$$

其中 a 是介于 0 和 1 之间的参数；$\min(\xi(x))$ 表示种子 x 所覆盖变量中最小的模糊测试次数；$n(v_i)$ 表示变量 v_i 的测试次数；t_x 表示从模糊测试任务开始到种子 x 被加入语料库的时间间隔；$\sum t_i$ 表示所有状态种子加入时间的总和。

我们倾向于选择测试次数较少的变量，因为这表明该变量尚未被充分测试。同时，我们也倾向于选择最近加入队列的种子，因为这表明该种子触发了新变量，或触发了较为罕见的变量状态。利用该种子选择概率，我们选出下一个更优的种子用于变异。

表 1: UNIBENCH 基准测试结果

项目	漏洞 ID	AFL		AFL++		FishFuzz		AFLGo		WindRanger		CmpLog		CSFuzz	
		TTR	TTE	TTR	TTE	TTR	TTE	TTR	TTE	TTR	TTE	TTR	TTE	TTR	TTE
jq	CVE-2015-8863	–	16h52m	–	33h20m	–	12h22m	–	8h19m	–	6h4m	–	31h10m	–	*5h47m
nm-new	CVE-2018-10373	T.O.	T.O.	43h3m	44h19m	40h32m	T.O.	T.O.	T.O.	T.O.	T.O.	23h4m	T.O.	*22h55m	*29h7m
nm-new	CVE-2018-12641	19h45m	T.O.	10h48m	T.O.	7h37m	T.O.	9h5m	T.O.	*3h46m	*26h5m	8h44m	T.O.	5h21m	37h1m
tcpdump	CVE-2017-5203	–	T.O.	–	39h28m	–	35h48m	–	44h40m	–	35h35m	–	34h39m	–	*33h42m
objdump	CVE-2017-9755	1h22m	2h40m	1h48m	8h20m	1h58m	3h19m	2h47m	5h5m	1h6m	*1h49m	1h23m	5h56m	*1h5m	2h43m
objdump	CVE-2017-9755	–	39h40m	–	38h3m	–	38h6m	–	36h13m	–	41h29m	–	*8h16m	–	33h41m
objdump	CVE-2017-7224	–	T.O.	–	T.O.	–	T.O.	–	*46h35m	–	T.O.	–	T.O.	–	T.O.
imginfo	CVE-2017-5500	T.O.	T.O.	T.O.	T.O.	28h27m	28h27m	T.O.	T.O.	T.O.	T.O.	44h35m	44h35m	*12h6m	*12h6m
wav2swf	CVE-2017-1000182	–	T.O.	–	T.O.	–	T.O.	–	T.O.	–	T.O.	–	T.O.	–	*12h42m
wav2swf	CVE-2017-11099	T.O.	T.O.	16h31m	T.O.	12h47m	T.O.	T.O.	T.O.	T.O.	T.O.	22h15m	T.O.	*9h51m	*11h49m
lame	CVE-2015-9101	17m	17m	24m	1h6m	16m	24m	15m	21m	12m	16m	27m	28m	*10m	*14m
lame	CVE-2017-11720	–	1h13m	–	1h16m	–	1h17m	–	1h9m	–	3h43m	–	55m	–	*51m
lame	CVE-2017-15046	11h21m	11h21m	7h38m	9h13m	12h41m	13h49m	7h12m	9h1m	15h	15h	*3h48m	*7h	6h55m	8h53m
sqlite3	CVE-2019-19646	T.O.	T.O.	T.O.	T.O.	*33h46m	*33h46m	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	35h25m	35h25m
sqlite3	CVE-2015-3416	T.O.	T.O.	T.O.	T.O.	*40h2m	*40h2m	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.
sqlite3	CVE-2019-19926	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	43h13m	43h13m	T.O.	T.O.	*39h52m	*39h52m
	平均加速比	+129%		+83%		+50%		+113%		+95%		+66%		/	
	平均漏洞数	4.7(+104%)		5.4(+78%)		6.6(+45%)		5.5(+74%)		6.4(+50%)		6.2(+54%)		9.6(/)	

4 实验与评估

CSFuzz 的主要组件包括静态分析与模糊测试两部分。在静态分析阶段，我们将源代码编译为 LLVM IR 文件，并在此基础上于 LLVM IR 层级执行静态分析以提取关键变量。关键变量的处理通过一个 LLVM Pass 实现，该模块由 1300 行 C++ 代码构成。在模糊测试阶段，CSFuzz 原型基于 AFL 2.57b 版本开发，通过 1200 行代码实现了核心组件功能。

表 2: MAGMA 基准测试结果

漏洞 ID	AFL		AFL++		FishFuzz		AFLGo		WindRanger		CmpLog		CSFuzz	
	TTR	TTE	TTR	TTE	TTR	TTE	TTR	TTE	TTR	TTE	TTR	TTE	TTR	TTE
XML009	–	T.O.	–	37h	–	7h41m	–	16h25m	–	10h52m	–	35h50m	–	*6h2m
XML017	–	*11m	–	50m	–	1h14m	–	45m	–	47m	–	15m	–	49m
SSL001	12m	9h15m	5m	6h56m	25m	29h	8m	8h35m	6m	6h33m	8m	8h58m	*4m	*5h22m
SND017	22m	*1h2m	18m	3h40m	27m	20h52m	15m	4h9m	10m	2h44m	14m	1h16m	*5m	1h48m
SND020	22m	T.O.	18m	4h56m	27m	9h12m	15m	5h6m	10m	3h11m	14m	6h38m	*5m	*1h23m
SQL002	13m	9h35m	9m	6h41m	41m	19h23m	9m	8h7m	5m	5h36m	5m	5h35m	6m	*4h19m
SQL014	23h44m	T.O.	14h12m	20h2m	22h53m	T.O.	15h47m	T.O.	*9h16m	*13h17m	12h43m	T.O.	10h24m	14h22m
SQL018	54m	10h38m	1h8m	13h21m	33m	*6h41m	42m	8h59m	*31m	7h51m	1h19m	15h13m	37m	6h54m
LUA003	T.O.	T.O.	T.O.	T.O.	*4h34m	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.
LUA004	30h12m	32h47m	20h25m	23h32m	32h22m	33h31m	33h41m	41h27m	26h29m	32h19m	24h13m	25h26m	*19h45m	*22h20m
TIF002	43h12m	43h12m	*36h16m	*36h16m	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	42h55m	42h57m	38h13m	38h24m
TIF007	2m	4m	2m	4m	3m	4m	7m	16m	12m	34m	2m	*2m	2m	4m
TIF008	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	42h22m	42h22m	*37h43m	*40h16m
TIF012	–	*49m	–	58m	–	1h20m	–	1h26m	–	4h9m	–	2h46m	–	1h28m
TIF014	2m	2h47m	2m	*58m	3m	5h21m	7m	2h14m	12m	4h39m	*1m	2h44m	2m	1h8m
PDF016	4m	12m	*1m	8m	4m	8m	3m	8m	4m	14m	2m	*2m	4m	8m
PDF010	15m	38m	23m	1h28m	7m	44m	11m	31m	9m	24m	9m	26m	*6m	*14m
PDF018	T.O.	T.O.	23h40m	23h40m	T.O.	T.O.	T.O.	T.O.	T.O.	T.O.	25h28m	25h28m	*22h46m	*22h46m
PDF021	4m	T.O.	3m	*25h30m	4m	T.O.	4m	T.O.	4m	T.O.	3m	33h12m	4m	T.O.
PDF006	13m	T.O.	12m	T.O.	15m	T.O.	11m	T.O.	12m	T.O.	*6m	T.O.	10m	*46h13m
平均加速比	+97%		+58%		+155%		+87%		+91%		+27%		/	
平均漏洞数	11(+40%)		14(+10%)		12(+28%)		13.3(+16%)		13.6(+13%)		14.1(+9%)		15.4(/)	

为验证 CSFuzz 的有效性，我们通过实验回答以下研究问题：

- RQ1：与其他模糊测试技术相比，CSFuzz 在漏洞复现方面的效果如何?
- RQ2：CSFuzz 的各组件对整体性能有何影响?
- RQ3：CSFuzz 的状态反馈是否会带来过高开销或状态爆炸问题?
- RQ4：CSFuzz 能否有效触发真实场景中的漏洞?

4.1 实验设置

对比方法。我们将 CSFuzz 与 6 种主流先进模糊测试工具进行对比。CSFuzz 为本文所提方法的原型，我们基于经验设置超参数并在实验中保持固定。AFL 是经典的边覆盖引导式灰盒模糊测试器，众多模糊测试工具均基于 AFL 框架实现。AFL++ 是持续更新的开源项目，整合了 AFL 系列多年来的优秀特性。FishFuzz 是当前最先进的定向灰盒模糊测试器之一，基于函数间调用距离引导模糊测试趋向目标位点并最大化代码覆盖。AFLGo 是定向灰盒模糊测试的开创性工作。WindRanger 利用偏差基本块辅助定向模糊测试，代表了最先进的定向模糊测试技术。AFL++ 的 CmpLog 模式受 REDQUEEN 启发，用于解决路径约束，但不针对特定目标位点。其余定向模糊测试工具在本文撰写时尚未开源。

实验数据集。我们采用 Unibench 数据集与 Magma 数据集。Unibench 数据集提供真实程序用于评估模糊测试器，可按照被测程序输入格式分为 6 类，我们选取覆盖所有类别的 8 个程序进行测试。Magma 数据集包含 9 个项目，通过向修复后的程序中手动植入已公开的 CVE 漏洞来评估模糊测试器性能。我们排除了两个无法编译的项目，并按照基准测试说明部署其余项目。

实验配置。为评估定向模糊测试器在漏洞复现中的性能，需要设置目标位点。对于 Unibench 项目，我们通过运行 MITRE CVE 数据库中的 PoC 输入获取目标位点；对于 Magma 数据集，我们使用其测试结果附件中的 PoC 输入。初始种子语料库的设置会影响实验有效性，为避免不一致性，Unibench 的初始语料库选自 AFL 提供的测试用例，若无对应文件格式，则使用满足被测程序输入的简单文件作为初始语料库。在 Magma 数据集上，我们选取基准测试提供的种子，删除导致程序崩溃的输入后作为初始语料库。我们在程序编译时添加 ASan 用于运行时检测，并关闭地址随机化以帮助 CSFuzz 更好地检测指针变量的运行时状态。默认每轮实验时间限制为 48 小时，共执行 10 轮并取结果平均值。实验环境为 Ubuntu 18.04 系统，搭载 Intel Xeon(R) Silver 4216 CPU 与 64GB 内存。在相同配置下，每个模糊测试器分配一个 CPU 核心在 Docker 容器中运行。

4.2 漏洞复现能力 (RQ1)

定向模糊测试的核心应用场景之一是高效复现漏洞。因此，我们在两个测试数据集上对比 CSFuzz 与 6 种先进模糊测试工具，采用目标到达时间（TTR）、漏洞触发时间（TTE）与平均触发漏洞数量作为评估指标。TTR 表示模糊测试器生成第一个到达目标的输入所需时间，TTE 表示触发漏洞所需时间。若所有模糊测试器均可在 10 分钟内触发漏洞，则排除该快速触发漏洞，因其无法验证模糊测试的有效性。

在 Unibench 数据集上，当漏洞被触发时，我们使用 ASan 记录的日志（崩溃代码行与最后三层调用栈）判断是否触发目标漏洞。Magma 数据集中的漏洞为手动植入，每个漏洞的触发由唯一 ID 标识。

Unibench 数据集结果如表 **??** 所示。符号“—”表示运行初始语料库即可到达对应目标位点，T.O. 表示模糊测试器在指定时间内未到达或触发对应漏洞。我们计算平均值得到 TTR 与 TTE，对于超时未触发的情况，使用 48 小时超时参数计算平均值，最短的平均 TTR 与 TTE 使用星号标记。

实验结果表明，CSFuzz 在大多数目标上均实现显著提升。与现有先进工具相比，CSFuzz 的 TTR 平均提升 89%，TTE 平均提升 95%，检测到的漏洞数量平均增加 67%。总体而言，相较于各对比工具，CSFuzz 在 TTR 上实现了不同幅度的加速，在漏洞发现数量上同样大幅领先。

Magma 数据集的评估结果如表 **??** 所示。CSFuzz 的 TTR 平均提升 85%，TTE 平均提升 115%，检测漏洞数量平均增加 19%，在各项指标上均优于对比工具。

CSFuzz 性能更优的原因在于：其他模糊测试技术无法捕获与目标相关的有效状态，可能因程序所需的特定数据条件而无法到达并触发漏洞。与缺乏额外反馈引导被测程序执行的模糊测试器不同，捕获相关变量状态的变化有助于到达并触发关联漏洞。

4.3 不同组件的影响（RQ2）

CSFuzz 的方法包括关键变量识别、关键变量触发的新程序状态感知、语料库选择与种子优先级区分。为研究不同组件对 CSFuzz 的影响，我们分别关闭每个组件并进行实验。

我们以默认配置的 CSFuzz 为基准，使用漏洞触发时间（TTE）作为评估指标。首先，我们关闭变量的自适应状态区间划分，使用统一的 2 的幂次对所有变量进行区间划分，记为 CSFuzz-T。其次，使用固定概率在状态语料库与代码覆盖语料库之间进行选择，记为 CSFuzz-C。然后，取消状态语料库内部的种子优先级排序，记为 CSFuzz-P。此外，关闭触发新程序状态的反馈，仅基于边覆盖引导模糊测试，记为 CSFuzz-S。最后，关闭距离信息，记为 CSFuzz-V，以评估关闭漏洞导向后的性能。

表 3: 各组件的影响

漏洞 ID	CSFuzz	CSFuzz-T	CSFuzz-C	CSFuzz-P	CSFuzz-S	CSFuzz-V
CVE-2015-8863	5h47m	8h29m	6h32m	7h24m	10h30m	6h24m
CVE-2018-10373	29h7m	37h22m	36h41m	37h33m	T.O.	T.O.
CVE-2018-12641	37h1m	41h5m	39h5m	40h43m	39h	39h27m
CVE-2017-5203	33h42m	26h41m	35h2m	34h22m	T.O.	38h30m
CVE-2017-9755	2h43m	9h8m	3h35m	4h22m	4h40m	3h51m
CVE-2017-9754	33h54m	39h26m	32h12m	35h15m	38h39m	35h50m
CVE-2017-5500	12h6m	17h56m	14h27m	15h12m	T.O.	17h13m
平均提升	/	+52%	+14%	+23%	+82%	+27%

如表 III 所示，默认的 CSFuzz 在 TTE 指标上分别优于 CSFuzz-T、CSFuzz-C、CSFuzz-P、CSFuzz-S、CSFuzz-V，性能提升分别为 52%、14%、23%、82%、27%。实验结果表明，CSFuzz-T 由于缺少自适应变量状态划分，在程序状态捕获方面存在不足。CSFuzz-C 与 CSFuzz-P 证明调度优化能够提升模糊测试性能。CSFuzz-S 表明关键变量的识别与状态检测对模糊测试具有显著贡献。关闭漏洞导向后，CSFuzz-V 的性能下降 27%，说明漏洞导向能够增强漏洞复现的整体效果。而关闭关键变量状态会导致 82% 的性能下降，这表明即使没有漏洞导向，关键变量状态依然能有效支撑定向模糊测试。这些组件共同构成了 CSFuzz，证明了监控关键变量状态在定向模糊测试中的有效性。

4.4 状态反馈的开耗（RQ3）

表 4: 运行时开销

项目	AFL	CSFuzz	开销
jq	72	65	10%
nm-new	159	153	10%
tcpdump	76	74	3%
objdump	155	125	19%
imginfo	356	338	5%
wav	208	192	8%
lame	38	31	18%
sqlite3	191	185	3%
平均	157	145	9%

CSFuzz 中状态反馈的实现需要额外插桩。为评估其执行速度开销，我们将其与 AFL 进行对比，因为本原型基于 AFL 实现。我们从状态语料库中选取种子作为输入以确保额外插桩代码被执行，并使用确定性模式保证测试用例一致。如表 IV 所示，中间两列整数为每秒平均执行次数，最后一列百分比为平均性能开销。结果表明，CSFuzz 引入的输入执行速度平均降低 9%，属于较低开销。CSFuzz 能够在性能损耗很小的情况下带来显著的效率提升。

表 5: 静态分析与状态反馈结果

项目	v-nums	s-nums	e-nums
sqlite3	313	433	6099
lua	533	1158	3190
tiff	331	667	3708
pdftoppm	255	367	8489
xmllint	593	586	3392
sndfile	344	693	1712
asn1	163	302	2122

除了性能开销，避免状态爆炸同样关键。在模糊测试过程中，我们维护两个语料库：覆盖新程序状态的状态队列与传统的边覆盖队列。表 V 展示了经过 48 小时模糊测试后，通过静态分析识别的关键变量数量以及不同语料库中存储的种子数量。其中，v-nums 表示识别的关键变量数量，s-nums 表示状态语料库种子数量，e-nums 表示边覆盖语料库种子数量。平均而言，状态语料库中的种子占总语料库的 16%，语料库增长较为温和，不会造成过多开销与状态爆炸。这是因为我们从程序中选取了与目标位点相关的关键变量，其中部分变量位于程序深处，可能不会被执行。此外，当测试用例同时触发新边与新状态时，仅将其加入边覆盖语料库而非状态语料库。最后，我们对关键变量的状态进行区间划分以避免状态爆炸。

表 6: 用于发现新漏洞的程序列表

程序	参数
w3m	@@
xmllint	–valid –dtdattr –stream @@
exiv2	@@
addr2line	-e @@
strip	-o /dev/null @@
podofopdfinfo	@@
ffmpeg	-i @@ test
freetype	@@
readelf	-w @@

4.5 真实场景评估（RQ4）

为验证 CSFuzz 是否有助于发现未知漏洞，我们将 CSFuzz、AFL、AFL++、FishFuzz、AFLgo、WindRanger 与 CmpLog 应用于 15 个最新版本的开源软件。我们收集了符合

目标格式的种子作为输入。对于需要目标的定向模糊测试，我们将软件中最近修复或修改的代码行设为目标位点。在相同配置下，每个模糊测试工具执行五次，每次持续七天。最终，CSFuzz 发现了四个独有的、此前未知的崩溃，而其他模糊测试工具未发现任何新漏洞。经过简要分析，我们将这些崩溃提交给开发者，其中三个已被快速确认并修复，gpac 与 libxml2 分别获得了 CVE-2024-32376 与 CVE-2024-32377。

我们以其中一个 CVE 为例简要说明 CSFuzz 如何触发漏洞。如列表 2 所示，CVE-2024-32376 源于使用非法格式文件导致的内存访问违规。在 `remain` 循环中，`remain` 表示待解析的字节数，`bytes_in_header` 表示 `hdr_store` 中的潜在字节数。当使用 `memmove` 复制更多字节时，如果 `MIN_HDR_STORE - bytes_in_header > remain`，就会发生堆缓冲区溢出。

Listing 2: 简化的 CVE-2024-32376 代码片段

```
GF_Err mpgviddmx_process(GF_Filter *filter)
{
    ...
    while (remain) {
        ...
        current = -1;
        if (ctx->bytes_in_header) {
            memmove(ctx->hdr_store + ctx->bytes_in_header,
                    start, MIN_HDR_STORE - ctx->bytes_in_header);
            current = mpgviddmx_next_start_code(ctx->hdr_store,
                                                MIN_HDR_STORE);
        }
        ...
    }
    ...
}
```

这段代码深度嵌入程序内部并受到多重约束，因此其他模糊测试工具即使执行到相关位置也无法捕获详细反馈，从而失去充分模糊测试的机会。CSFuzz 跟踪 `bytes_in_header` 与 `remain` 变量的数值变化，并在目标附近分配资源进行充分模糊测试，从而触发漏洞。

5 有效性威胁

在实验中，我们基于实践经验为 CSFuzz 选取了超参数设置。由于参数调优需要投入大量时间与资源，且当前实验效果已较为理想，因此我们将完善可配置选项作为未来工作的一部分。

此外，CSFuzz 中的状态反馈机制需要对目标位置附近的关键变量进行监控。当目标位置难以被覆盖、且缺少邻近的种子执行样本时，这些目标无法获得相应的状态反馈，从而导致 CSFuzz 难以进一步提供有效的测试引导。

6 相关工作

我们的研究涉及以下研究领域：

定向灰盒模糊测试 (Directed Grey-box Fuzzing, DGF)。DGF 在针对性崩溃复现和补丁测试中特别有效。AFLGo [15] 首次引入了定向模糊测试的概念。在运行时，AFLGo 根据种子到目标点的调和平均距离来优先处理种子。Hawkeye [19] 最初提出在函数层级测量种子执行轨迹与目标执行轨迹之间的相似性，并以此作为种子优先级划分的标准。BEACON [34] 利用静态分析来终止无法到达任何目标点的场景。WindRanger [21] 通过考虑偏离目标路径的基本块，改进了种子到目标点的距离计算。Fish Fuzz [20] 通过使用每个种子的函数间距离，避免了距离计算的不准确性。DAFL [40] 执行数据流分析，根据数据流语义计算出的距离来优先处理种子。

这些方法有助于到达目标点，但没有考虑用于彻底测试的额外反馈。相比之下，CSFuzz 通过考虑关键变量的状态，对目标点进行更全面的测试，这有助于触发目标漏洞。此外，我们的方法与这些方法是正交的，我们可以采用它们来有效地促进到达目标区域附近。

基于污点分析的模糊测试 (Taint-analysis-based Fuzzing)。污点分析也被广泛用于增强模糊测试。Angora [41] 使用动态污点分析来推断与路径约束相关的输入字节形状，从而确定在何处以及如何对输入进行变异。REDQUEEN [35] 观察比较指令中使用的值，然后使用随机变异对输入进行染色，以推断与输入直接复制相关的污点。GREYONE [32] 采用动态污点分析来指导变异过程并优先处理种子。

虽然这些方法引导了变异并有助于解决路径约束，但它们主要关注影响这些约束的变量，而忽略了其他变量。相比之下，我们考虑了与目标相关的关键变量的状态反馈。此外，我们的方法可以结合污点分析技术来增强变异，这与现有方法并不冲突。

覆盖率指导的灰盒模糊测试 (Coverage-guided Grey-box Fuzzing)。边覆盖率（Edge coverage）是一种可以有效指导模糊测试的覆盖反馈形式。近期的研究提出使用上下文敏感的边覆盖率 [41] 或路径覆盖率 [39, 42] 来增强边覆盖率。此外，还有一些方法探索使用数据信息作为补充。INVSCOV [43] 通过预执行收集程序不变式，并整合了不变式覆盖反馈。DDFuzz [44] 和 DATAFLOW [45] 引入了数据流边，以探索传统控制流边之外的覆盖反馈。

然而，单纯增加覆盖率信号意味着更多的输入被视为有价值的，这可能会由于无差别的测试而降低模糊测试器的速度。虽然 Ijon [46] 通过使用程序中标记的重要注释改进了覆盖率度量，但它需要专家对程序有深入的理解来进行手动标记。相比之下，CSFuzz 将问题陈述转向定向模糊测试，专注于来自更多关键变量的状态覆盖反馈。这种方法避免了状态爆炸，并在有效性和效率之间取得了平衡，从而实现了更有效的漏洞发现。

7 结论

本文提出监控关键变量状态的思想，并实现了定向灰盒模糊测试原型系统 CSFuzz。该系统能够识别与目标位置相关的关键变量，并在运行时保留触发新状态的种子。我们

在基准测试程序与实际程序上对 CSFuzz 进行了实验评估。结果表明，CSFuzz 能够比现有先进模糊测试工具更快地暴露漏洞。值得注意的是，CSFuzz 在两款经过广泛测试的程序中发现了两个 0-day 漏洞，证明了其实用价值。

致谢

感谢匿名审稿人提出的深刻意见与建议，对本文改进具有重要帮助。同时感谢韩晓贝、张志杰、魏浩来对本文工作的讨论。本工作得到国家自然科学基金（No. 62172407）与中国科学院青年创新促进会资助。

参考文献

[1] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "Afl++: Combining incremental steps of fuzzing research," in *14th USENIX Conference on Offensive Technologies (WOOT'20)*. USENIX Association, 2020.

[2] G. Inc, "Syzkaller-kernel fuzzer," https://github.com/google/syzkaller, 2016, accessed July 22, 2024.

[3] K. Serebryany, "Libfuzzer: A library for coverage-guided fuzz testing (within llvm)," https://llvm.org/docs/LibFuzzer.html, 2017, accessed July 22, 2024.

[4] M. Zalewski, "American fuzzy lop," http://lcamtuf.coredump.cx/afl/, 2014, accessed July 22, 2024.

[5] S. Gan, C. Zhang, X. Qin, X. Tu, K. Li, Z. Pei, and Z. Chen, "Collafl: Path sensitive fuzzing," in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 679–696.

[6] C. Lemieux and K. Sen, "Fairfuzz: A targeted mutation strategy for increasing greybox fuzz testing coverage," in *33rd ACM/IEEE International Conference on Automated Software Engineering (ASE ' 18)*. ACM, 2018, pp. 475–485.

[7] C. Lyu, S. Ji, C. Zhang, Y. Li, W.-H. Lee, Y. Song, and R. Beyah, "Mopt: Optimized mutation scheduling for fuzzers," in *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, 2019, pp. 1949–1966.

[8] W. You, X. Wang, S. Ma, J. Huang, X. Zhang, X. Wang, and B. Liang, "Profuzzer: On-the-fly input type probing for better zero-day vulnerability discovery," in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 769–786.

[9] K. Serebryany, "Oss-fuzz - google's continuous fuzzing service for open source software," in *USENIX Association*, 2017.

[10] M. Woo, S. K. Cha, S. Gottlieb, and D. Brumley, "Scheduling blackbox mutational fuzzing," in *2013 ACM SIGSAC Conference on Computer & Communications Security (CCS ' 13)*. ACM, 2013, pp. 511–522.

[11] V. Ganesh, T. Leek, and M. Rinard, "Taint-based directed whitebox fuzzing," in *2009 IEEE 31st International Conference on Software Engineering*. IEEE, 2009, pp. 474–484.

[12] P. Godefroid, A. Kiezun, and M. Y. Levin, "Grammar-based whitebox fuzzing," in *29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI ' 08)*. ACM, 2008, pp. 206–215.

[13] V.-T. Pham, M. Böhme, and A. Roychoudhury, "Model-based whitebox fuzzing for program binaries," in *31st IEEE/ACM International Conference on Automated Software Engineering (ASE ' 16)*. ACM, 2016, pp. 543–553.

[14] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based greybox fuzzing as markov chain," in *2016 ACM SIGSAC Conference on Computer and Communications Security (CCS ' 16)*. ACM, 2016, pp. 1032–1043.

[15] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in *2017 ACM SIGSAC Conference on Computer and Communications Security (CCS ' 17)*. ACM, 2017, pp. 2329–2344.

[16] P. Godefroid, N. Klarlund, and K. Sen, "Dart: Directed automated random testing," in *2005 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI ' 05)*. ACM, 2005, pp. 213–223.

[17] M. Christakis, P. Müller, and V. Wüstholz, "Guiding dynamic symbolic execution toward unverified program executions," in *38th International Conference on Software Engineering (ICSE ' 16)*. ACM, 2016, pp. 144–155.

[18] J. Xuan, X. Xie, and M. Monperrus, "Crash reproduction via test case mutation: Let existing test cases help," in *2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2015)*. ACM, 2015, pp. 910–913.

[19] H. Chen, Y. Xue, Y. Li, B. Chen, X. Xie, X. Wu, and Y. Liu, "Hawkeye: Towards a desired directed grey-box fuzzer," in *2018 ACM SIGSAC Conference on Computer and Communications Security (CCS ' 18)*. ACM, 2018, pp. 2095–2108.

[20] H. Zheng, J. Zhang, Y. Huang, Z. Ren, H. Wang, C. Cao, Y. Zhang, F. Toffalini, and M. Payer, "Fishfuzz: Catch deeper bugs by throwing larger nets," in *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, 2023, pp. 1343–1360.

[21] Z. Du, Y. Li, Y. Liu, and B. Mao, "Windranger: A directed greybox fuzzer driven by deviation basic blocks," in *44th International Conference on Software Engineering (ICSE ' 22)*. ACM, 2022, pp. 2440–2451.

[22] H. Huang, P. Yao, H. CHIU, Y. Guo, and C. Zhang, "Titan: Efficient multi-target directed greybox fuzzing," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 62–62.

[23] H. Rong, W. You, X. Wang, and T. Mao, "Toward unbiased multiple-target fuzzing with path diversity," Oct 2023.

[24] S. Inc, "Heartbleed," https://heartbleed.com/, 2020, accessed July 22, 2024.

[25] Taviso, "Cloudbleed," https://bugs.chromium.org/p/project-zero/issues/detail?id=1139, 2017, accessed July 22, 2024.

[26] K. Serebryany, "A memory error detector," https://github.com/google/sanitizers/wiki/AddressSanitizer, 2012, accessed July 22, 2024.

[27] Y. Li, S. Ji, Y. Chen, S. Liang, W.-H. Lee, Y. Chen, C. Lyu, C. Wu, R. Beyah, P. Cheng, K. Lu, and T. Wang, "Unifuzz: A holistic and pragmatic metrics-driven platform for evaluating fuzzers," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2021, pp. 2777–2794.

[28] A. Hazimeh, A. Herrera, and M. Payer, "Magma: A ground-truth fuzzing benchmark," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 4, no. 3, Nov 2020.

[29] Gpac, "An open-source multimedia framework," https://github.com/gpac/gpac, 2000, accessed July 22, 2024.

[30] Libxml2, "Gnome project," https://github.com/GNOME/libxml2/, 1999, accessed July 22, 2024.

[31] Y. Sui and J. Xue, "Svf: interprocedural static value-flow analysis in llvm," in *25th International Conference on Compiler Construction (CC ' 16)*. ACM, 2016, pp. 265–266.

[32] S. Gan, C. Zhang, P. Chen, B. Zhao, X. Qin, D. Wu, and Z. Chen, "Greyone: Data flow sensitive fuzzing," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 2020, pp. 2577–2594.

[33] I. Yun, S. Lee, M. Xu, Y. Jang, and T. Kim, "Qsym : A practical concolic execution engine tailored for hybrid fuzzing," in *27th USENIX Security Symposium (USENIX Security 18)*. USENIX Association, 2018, pp. 745–761.

[34] H. Huang, Y. Guo, Q. Shi, P. Yao, R. Wu, and C. Zhang, "Beacon: Directed grey-box fuzzing with provable path pruning," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 36–50.

[35] C. Aschermann, S. Schumilo, T. Blazytko, R. Gawlik, and T. Holz, "Redqueen: Fuzzing with input-to-state correspondence," in *Proceedings 2019 Network and Distributed System Security Symposium*, 2019.

[36] M. Corporation, "Mitre cve database," https://cve.mitre.org/, 1958, accessed July 22, 2024.

[37] A. Hazimeh, "Magma artifacts," https://osf.io/resj8/, 2020, accessed July 22, 2024.

[38] Docker, "Use containers to build, share and run your applications," https://www.docker.com/resources/what-container, 2021, accessed July 22, 2024.

[39] J. Wang, Y. Duan, W. Song, H. Yin, and C. Song, "Be sensitive and collaborative: Analyzing impact of coverage metrics in greybox fuzzing," in *22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019)*. USENIX Association, 2019, pp. 1–15.

[40] T. E. Kim, J. Choi, K. Heo, and S. K. Cha, "Dafl: Directed grey-box fuzzing guided by data dependency," in *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, 2023, pp. 4931–4948.

[41] P. Chen and H. Chen, "Angora: Efficient fuzzing by principled search," in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 711–725.

[42] S. Yan, C. Wu, H. Li, W. Shao, and C. Jia, "Pathafl: Path-coverage assisted fuzzing," in *15th ACM Asia Conference on Computer and Communications Security (ASIA CCS ' 20)*. ACM, 2020, pp. 598–609.

[43] A. Fioraldi, D. C. D' Elia, and D. Balzarotti, "The use of likely invariants as feedback for fuzzers," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2021, pp. 2829–2846.

[44] A. Mantovani, A. Fioraldi, and D. Balzarotti, "Fuzzing with data dependency information," in *2022 IEEE 7th European Symposium on Security and Privacy (EuroSP)*. IEEE, 2022, pp. 286–302.

[45] A. Herrera, M. Payer, and A. L. Hosking, "Dataflow: Toward a data-flow-guided fuzzer," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 5, Jul 2023.

[46] C. Aschermann, S. Schumilo, A. Abbasi, and T. Holz, "Ijon: Exploring deep state spaces via fuzzing," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 1597–1612.